

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	M.HASHETHA	Reg. No.:	192521162
Date:	13.08.2026	Duration:	60 minutes

Code of Conduct

I M.HASHETHA(192521162) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.HASHETHA

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Analysis

1.1 Problem Statement

The current court case management process can involve large amounts of manual documentation, case tracking, scheduling, and administrative work. This can lead to delays in case processing, inefficient hearing schedules, difficulty in managing judicial workloads, and limited access to useful case analytics. The proposed AI-Powered Smart Court Case Management and Judicial Analytics Platform is designed to digitize these activities. The platform provides centralized case management, legal document classification, hearing scheduling, case-duration prediction, judicial workload analysis, and analytics dashboards.

The system aims to support court administrators and judicial authorities by providing organized information and data-driven insights, while improving transparency and accessibility.

1.2 Project Objectives

The main objectives are:

1. Digitize court case management and maintain case information electronically.
2. Automate legal document classification using AI/NLP techniques.
3. Predict case completion timelines based on available case information.
4. Optimize hearing schedules to reduce scheduling conflicts and delays.
5. Analyze judicial workload using case and hearing data.
6. Generate legal analytics reports for administrative decision-making.
7. Improve transparency by providing organized case information and dashboards.
8. Reduce case backlog through better case tracking and scheduling.

1.3 Functional Requirements

These describe **what the system should do**.

FR1 — User Management

The system should allow authorized users to log in and access functions according to their roles.

Possible roles:

- Administrator
- Judge
- Court Staff

FR2 — Case Management

The system should allow authorized users to:

- Create a new case
- View case details
- Update case information
- Track case status
- Search for cases
- Archive completed cases

FR3 — Document Management

The system should:

- Upload legal documents
- Store document metadata
- Associate documents with cases
- Classify documents into categories

FR4 — Hearing Management

The system should:

- Schedule hearings
- View upcoming hearings
- Assign judges
- Assign courtrooms
- Detect scheduling conflicts

FR5 — AI-Based Document Classification

The system should analyze document content and classify documents into appropriate categories.

FR6 — Case Duration Prediction

The system should provide an estimated case completion timeline using historical and case-related.

FR7 — Judicial Analytics

The system should display:

- Total cases
- Pending cases
- Completed cases
- Cases by category
- Cases handled by each judge

2.Project Structure Design

Proposed Repository Structure

smart-court-management

```
|
|
|— frontend/
|   |— index.html
|   |— cases.html
|   |— hearings.html
|   |— analytics.html
|   |— style.css
|   └— script.js
|— backend/
|   |— app.py
|   |— requirements.txt
|   |— routes/
|   |   |— case_routes.py
|   |   |— hearing_routes.py
|   |   └— document_routes.py
|   └— services/
|       |— case_service.py
|       └— hearing_service.py
|— ai-service/
|   |— app.py
|   |— classifier.py
|   └— predictor.py
|— database/
|   └— init.sql
|— docs/
|   |— architecture.png
|   └— workflow.png
|— screenshots/
|   |— git/
|   └— docker/
```

```
|— .gitignore
|— Dockerfile
|— docker-compose.yml
|— README.md
└— LICENSE
```

3. Git Repository Initialization

For our proposed **AI-Powered Smart Court Case Management and Judicial Analytics Platform**, Git is used as the **version control system** to track changes in the project and support collaborative development.

Since we're **not actually implementing the project**, we only need to document the initialization process and commands.

3.1 Purpose of Git

Git helps the development team:

- Track changes made to project files.
- Maintain different versions of the project.
- Create separate branches for different features.
- Merge changes from different developers.
- Recover previous versions when required.
- Collaborate through a remote repository such as GitHub.

3.2 Git Repository Initialization

The proposed project directory is:

```
smart-court-management/
```

The repository can be initialized using:

git init

This converts the project folder into a Git repository.

Then the project files can be checked using:

git status

Files can be added to the staging area using:

git add .

The initial version can then be committed using:

git commit -m "Initial project structure"

3.3 Essential Git Files

After executing:

```
git init
```

Git creates a hidden directory:

```
.git/
```

The .git directory contains the internal information required by Git to track the project history.

We don't normally modify the contents of .git manually.

Another important file is:

```
.gitignore
```

It specifies files and folders that should **not be tracked by Git**.

This prevents unnecessary files and sensitive configuration information from being committed.

4. Git Branching Strategy

A Git Flow-based branching strategy is proposed for the AI-Powered Smart Court Case Management and Judicial Analytics Platform. It helps manage different development activities and keeps the main project stable.

4.1 Proposed Branches

1. Main Branch

The main branch contains the stable and tested version of the project. Only completed and approved changes are merged into this branch.

2. Development Branch

The development branch is used to combine and test different features before they are released to the main branch.

3. Feature Branch

Feature branches are created for developing individual modules separately.

Examples:

- feature/case-management
- feature/hearing-scheduling
- feature/document-classification
- feature/judicial-analytics

After completion, feature branches are merged into the development branch.

4. Release Branch

A release branch is created when the project is ready for final testing and release.

Example: release/v1.0

It is used for final testing, bug fixing, and documentation.

5. Hotfix Branch

A hotfix branch is created to fix urgent problems found in the stable main branch.

Example: hotfix/login-error

After fixing the issue, the hotfix is merged back into the main branch.

4.2 Branching Workflow

The proposed workflow is:

Feature Branch → Development → Release → Main

For urgent issues:

Main → Hotfix → Main

4.3 Justification

This branching strategy is suitable because the project contains multiple modules such as case management, hearing scheduling, document classification, and judicial analytics. Developers can work on separate features without affecting the stable version.

The strategy provides better collaboration, controlled development, easier testing, and safer releases. It also allows urgent problems to be fixed using hotfix branches.

5. Version Control Workflow

Git is used to manage and track changes made to the project. A structured workflow is proposed to ensure that changes are developed, reviewed, and merged safely.

5.1 Basic Git Workflow

The proposed workflow is:

Create Branch → Make Changes → Stage Changes → Commit → Merge → Push

5.2 Git Operations

1. Create a Branch

A feature branch is created for developing a particular feature.

`git checkout -b feature/case-management`

2. Stage Changes

The modified files are added to the staging area.

`git add .`

3. Commit Changes

The changes are saved with a meaningful commit message.

`git commit -m "Add case management module"`

4. Merge Changes

After completing and reviewing a feature, it is merged into the development branch.

git checkout development

git merge feature/case-management

5. Synchronize Repository

The changes are uploaded to the remote GitHub repository.

git push origin development

5.3 Conflict Resolution

A merge conflict may occur when two branches modify the same part of a file. The conflicting changes are reviewed and the correct version is selected. After resolving the conflict, the changes are committed again.

git add .

git commit -m "Resolve merge conflict"

5.4 Purpose of Git Operations

Git operations help maintain different versions of the project, support teamwork, prevent accidental loss of work, and provide a clear history of project development.

The proposed workflow ensures that features are developed separately, tested before merging, and synchronized with the remote repository.

6. Commit History Analysis

Git maintains a complete history of the changes made to a project. Every important modification can be stored as a commit with a meaningful message. This helps developers understand how the project has developed over time and makes it easier to identify and manage changes.

For the proposed AI-Powered Smart Court Case Management and Judicial Analytics Platform, Git commits are organized according to major project activities such as project setup, feature development, database configuration, Docker configuration, and documentation.

6.1 Sample Commit History

The proposed commit history for the project is:

Initial project structure

Add case management module

Add hearing scheduling module

Add document classification module

Add judicial analytics module

Add database configuration

Add user management module

Add Docker configuration

Add Docker Compose configuration

Update project documentation

Fix application configuration

Update system architecture documentation

Each commit represents a specific development activity. This makes the project history organized and easy to understand.

6.2 Meaningful Commit Messages

Commit messages should clearly describe the change made in the project. A good commit message should be short, specific, and easy for other developers to understand.

For example:

```
git commit -m "Add hearing scheduling module"
```

This message clearly indicates that a hearing scheduling feature has been added.

A message such as:

```
git commit -m "changes"
```

is not recommended because it does not explain what was changed.

6.3 Good Commit Practices

The following practices are proposed for maintaining a professional commit history:

- Use short and descriptive commit messages.
- Make each commit related to a specific change.
- Avoid unnecessary or duplicate commits.
- Use consistent wording throughout the project.
- Commit completed and meaningful changes.
- Avoid including unrelated modifications in a single commit.
- Review the changes before committing them.

These practices make the repository easier to understand and maintain.

6.4 Example Development History

The development of the proposed system can be divided into different stages.

Stage 1 – Project Setup

Initial project structure

This commit establishes the basic repository structure.

Stage 2 – Core Features

Add case management module

Add user management module

Add hearing scheduling module

These commits represent the development of the main system features.

Stage 3 – AI and Analytics

Add document classification module

Add judicial analytics module

These commits represent the proposed AI and analytics functionality.

Stage 4 – Deployment Configuration

Add Docker configuration

Add Docker Compose configuration

These commits represent the containerization and deployment configuration.

Stage 5 – Documentation

Update project documentation

Update system architecture documentation

These commits represent updates to the technical documentation.

6.5 Importance of Commit History

A clear Git commit history provides several benefits:

1. It helps track the development progress of the project.
2. It shows what changes were made at different stages.
3. It helps developers understand the purpose of each modification.
4. It makes troubleshooting and debugging easier.
5. It allows previous versions of the project to be identified.
6. It supports collaboration between team members.
7. It provides a record of the development process.
8. It makes project maintenance easier.

6.6 Commit History and Project Maintenance

The commit history acts as a record of the project's development. If a problem is discovered after a particular change, developers can examine the commit history to identify the change that introduced the problem.

For example, if an issue occurs after adding the hearing scheduling module, the development team can review the related commit and investigate the changes associated with it.

7. Docker Environment Design

Docker is a containerization platform that packages an application and its required dependencies into isolated containers. Containers allow applications to run consistently across different environments without requiring the same software configuration on every system.

For the proposed AI-Powered Smart Court Case Management and Judicial Analytics Platform, Docker provides a suitable environment for organizing the different components of the application.

7.1 Need for Docker

The proposed platform contains multiple components such as the frontend, backend, AI and analytics service, and database. Managing these components directly on a computer may require installing and configuring several software packages.

Docker simplifies this process by allowing each major component to run in its own container.

The main benefits of using Docker for this project are:

- Provides a consistent development environment.
- Reduces dependency and configuration problems.
- Separates different application components.
- Makes deployment easier.
- Improves portability between systems.
- Supports easier scaling of individual services.
- Makes application management more organized.

7.2 Frontend Container

The frontend container provides the user interface of the system.

It contains the web pages used by administrators, judges, and court staff.

Main responsibilities include:

- Displaying the dashboard.
- Providing case management screens.
- Displaying hearing information.
- Showing analytics.
- Sending requests to the backend.

The frontend communicates with the backend through the application network.

7.4 Backend Container

The backend container provides the main application logic and APIs.

Its responsibilities include:

- Managing court cases.

- Managing users.
- Managing hearings.
- Processing application requests.
- Communicating with the database.
- Communicating with the AI and analytics service.

The backend acts as the main connection between the frontend, database, and AI service.

7.5 AI and Analytics Container

The AI service is designed to handle the intelligent features of the proposed platform.

Its responsibilities include:

- Legal document classification.
- Case duration estimation.
- Judicial workload analysis.
- Processing information required for analytics.

Keeping the AI service separate from the backend improves modularity and allows the AI component to be updated independently.

7.6 Database Container

A MySQL database container is proposed for storing the application's data.

The database can contain information related to:

- Users.
- Court cases.
- Legal documents.
- Hearings.
- Case status.
- Analytics information.

A Docker volume can be used to preserve database information even when the database container is stopped or recreated.

7.7 Docker Network

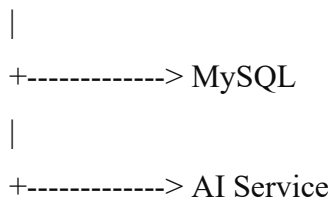
The containers communicate through a common Docker network.

The proposed communication is:

Frontend

|
v

Backend



The Docker network allows the services to communicate with each other without exposing every internal service directly to the outside network.

7.8 Docker Volume

A persistent volume is proposed for the database.

MySQL Container



Docker Volume



Persistent Database Data

The volume ensures that important database information is not lost when the database container is restarted or recreated.

7.9 Environment Variables

Environment variables can be used to store configuration information such as:

- Database host.
- Database port.
- Database name.
- Database username.
- Database password.
- Backend service URL.

Sensitive information such as database passwords should not be directly written inside application source code.

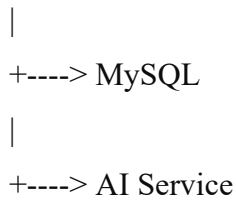
7.10 Service Dependencies

The proposed services have the following dependencies:

Frontend



Backend



The backend depends on the database and AI service for certain operations. The frontend depends on the backend to obtain application data.

Docker Compose can be used to define these services and manage their relationships.

7.11 Why Docker is Suitable

Docker is suitable for this project because the platform consists of multiple independent components. Each component can be packaged and managed separately while still communicating through a common network.

Docker also improves portability because the same container configuration can be used across different development and deployment environments.

It supports maintainability by keeping application components isolated and makes it easier to update or scale individual services when required.

7.12 Summary

The proposed Docker environment consists of frontend, backend, AI and analytics, and database containers. Docker networking enables communication between these services, while Docker volumes provide persistent storage for database information. Environment variables provide flexible configuration, and Docker Compose can be used to manage the complete environment.

Therefore, Docker provides a portable, organized, and maintainable deployment environment for the proposed Smart Court Case Management and Judicial Analytics Platform.

8. Dockerfile Development

A Dockerfile is a text file that contains a set of instructions used to create a Docker image. The image contains the application and the required software dependencies needed to run it.

For the proposed Smart Court Case Management and Judicial Analytics Platform, a Dockerfile can be used to define the environment required for the backend application.

8.1 Proposed Dockerfile

```
FROM python:3.12-slim
```

```
WORKDIR /app
```


`COPY requirements.txt .`

`RUN pip install --no-cache-dir -r requirements.txt`

`COPY . .`

`EXPOSE 5000`

`CMD ["python", "app.py"]`

8.2 Explanation of Dockerfile Instructions

FROM

`FROM python:3.12-slim`

The FROM instruction specifies the base image. A lightweight Python image is selected because the backend and analytics components are proposed to use Python.

WORKDIR

`WORKDIR /app`

The WORKDIR instruction sets /app as the working directory inside the container. Application files and commands are executed from this directory.

COPY

`COPY requirements.txt .`

This copies the dependency file into the container so that the required software packages can be installed.

RUN

`RUN pip install --no-cache-dir -r requirements.txt`

The RUN instruction installs the Python packages required by the application.

COPY

`COPY . .`

This copies the application source files into the container.

EXPOSE

`EXPOSE 5000`

The EXPOSE instruction indicates that the application uses port 5000 for communication.

CMD

```
CMD ["python", "app.py"]
```

The CMD instruction specifies the command that starts the application when the container is launched.

8.3 Dockerfile Workflow

The Dockerfile follows this general process:

Base Image



Set Working Directory



Copy Dependencies



Install Dependencies



Copy Application Files



Expose Application Port



Start Application

8.4 Benefits of the Dockerfile

The Dockerfile provides several benefits:

1. Creates a consistent application environment.
2. Automatically installs required dependencies.
3. Reduces manual configuration.
4. Makes the application portable.
5. Allows the same environment to be recreated when required.
6. Simplifies deployment using Docker.
7. Supports integration with Docker Compose.

8.5 Summary

The proposed Dockerfile defines the environment required to run the backend application. It selects a Python base image, installs the required dependencies, copies the application files, exposes the required port, and defines the application startup command.

This provides a standardized and reproducible environment for deploying the proposed Smart Court Case Management and Judicial Analytics Platform.

9. Docker Compose Design

Docker Compose is a tool used to define and manage multiple Docker containers as a single application. It allows different services to be configured in one file and started together.

For the proposed AI-Powered Smart Court Case Management and Judicial Analytics Platform, Docker Compose is used to define the frontend, backend, AI service, and database services.

9.1 Proposed Docker Compose Configuration

services:

frontend:

build: ./frontend

ports:

- "3000:3000"

depends_on:

- backend

backend:

build: ./backend

ports:

- "5000:5000"

depends_on:

- database

- ai-service

ai-service:

build: ./ai-service

ports:

- "6000:6000"

database:

image: mysql:8.0

environment:

MYSQL_DATABASE: court_db

MYSQL_USER: court_user

MYSQL_PASSWORD: court_password

MYSQL_ROOT_PASSWORD: root_password

volumes:

- court_data:/var/lib/mysql

volumes:

court_data:

9.2 Services

The Docker Compose configuration contains four main services.

Frontend

The frontend service provides the user interface of the application. It communicates with the backend service to request and display information.

Backend

The backend service manages the main application logic. It communicates with the database and AI service.

AI Service

The AI service provides document classification, prediction, and analytics-related functionality.

Database

The database service uses MySQL to store court case, user, document, and hearing information.

9.3 Port Mapping

Ports are used to allow communication between the host system and the containers.

The proposed ports are:

Service	Port
---------	------

Frontend	3000
----------	------

Backend	5000
---------	------

AI Service	6000
------------	------

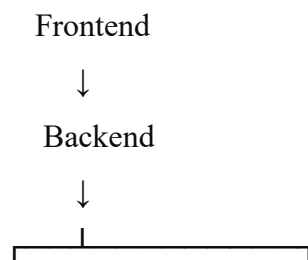
MySQL	3306
-------	------

Port mapping allows users and applications to access the required services.

9.4 Networking

Docker Compose automatically creates a network for the services defined in the configuration.

The communication can be represented as:





The services can communicate with each other using their service names within the Docker network.

9.5 Volumes

A volume is used for persistent database storage.

`court_data:/var/lib/mysql`

The `court_data` volume ensures that database information remains available even if the MySQL container is stopped or recreated.

9.6 Environment Variables

Environment variables are used to configure the database.

Examples include:

- Database name
- Database username
- Database password

This provides flexible configuration and avoids hard-coding configuration values directly into the application.

9.7 Service Dependencies

The `depends_on` option defines basic service startup dependencies.

The proposed dependency structure is:

Frontend



Backend



Database



AI Service

The backend requires the database and AI service for its operations, while the frontend requires the backend to provide application data.

9.8 Advantages of Docker Compose

Docker Compose provides the following benefits:

1. Manages multiple containers together.
2. Simplifies application deployment.

3. Provides centralized configuration.
4. Makes service communication easier.
5. Supports persistent storage through volumes.
6. Allows environment variables to be defined centrally.
7. Reduces manual container management.
8. Makes the development environment easier to reproduce.

9.9 Summary

The proposed Docker Compose configuration organizes the Smart Court platform into separate frontend, backend, AI service, and database containers. Networking allows these services to communicate, while volumes provide persistent database storage. Environment variables provide configuration, and service dependencies define the relationship between components.

Therefore, Docker Compose provides a simple and organized method for managing the complete containerized environment.

10. Container Deployment and Testing

Docker provides a simple method for deploying the different components of the proposed Smart Court Case Management and Judicial Analytics Platform. Docker Compose can be used to build and start all the required services together.

10.1 Deployment Process

The proposed deployment process consists of the following steps:

1. Prepare the project repository.
2. Create the required Dockerfiles.
3. Configure the Docker Compose file.
4. Build the required Docker images.
5. Start the containers.
6. Check the container status.
7. Test communication between services.
8. Access the application through the browser.

10.2 Build the Docker Environment

The Docker images can be built using:

`docker compose build`

This command reads the Docker Compose configuration and builds the required images for the application services.

10.3 Start the Containers

The complete application environment can be started using:

`docker compose up`

This starts the services defined in the Docker Compose configuration.

The proposed services are:

- Frontend
- Backend
- AI Service
- MySQL Database

10.4 Check Running Containers

The following command can be used to check the running containers:

`docker compose ps`

The expected result should show the configured services in a running state.

10.5 Application Testing

After starting the containers, the application can be accessed through the configured frontend port.

For example:

`http://localhost:3000`

The following functions can be considered for testing:

Test	Expected Result
Open application	Dashboard should be displayed
Add case	Case information should be accepted
View cases	Stored cases should be displayed
Schedule hearing	Hearing information should be recorded
Upload document	Document information should be processed
Document classification	Appropriate category should be displayed
View analytics	Case statistics should be displayed

10.6 Service Communication Testing

The communication between services should also be verified.

Frontend → Backend

Backend → Database

Backend → AI Service

Successful communication indicates that the Docker network and service configuration are working correctly.

10.7 Stopping the Containers

After testing, the containers can be stopped using:

`docker compose down`

This stops and removes the containers created by Docker Compose.

10.8 Testing Evidence

For the final technical report, the following evidence can be included if the environment is actually executed:

- Docker image build screenshot.
- Docker Compose execution screenshot.
- Running container status screenshot.
- Application dashboard screenshot.
- Case management screen screenshot.
- Analytics screen screenshot.

Since this project is being presented as a proposed design, these commands represent the intended deployment and testing procedure.

10.9 Expected Result

The expected result is that all required services start successfully and communicate through the Docker network. The frontend should be able to communicate with the backend, while the backend should be able to access the database and AI service.

The proposed containerized deployment provides a consistent environment for running the Smart Court Case Management and Judicial Analytics Platform.

11. Benefits of Git and Docker

Git and Docker provide important benefits for the development and deployment of the proposed AI-Powered Smart Court Case Management and Judicial Analytics Platform. Git manages the source code and project history, while Docker provides a consistent environment for running the application.

11.1 Benefits of Git

1. Version Management

Git maintains different versions of the project and allows developers to track changes made over time.

2. Collaboration

Multiple developers can work on different features using separate branches and later merge their changes.

3. Branch Management

Feature, development, release, and hotfix branches help organize the development process.

4. Change Tracking

Git records changes through commits, making it easier to identify what was modified and when.

5. Backup and Recovery

A remote Git repository provides an additional copy of the project and allows previous versions to be recovered when required.

6. Project Maintenance

A clear commit history helps developers understand the development process and troubleshoot problems.

11.2 Benefits of Docker

1. Portability

Docker allows the application to run consistently across different environments.

2. Consistent Environment

The required software dependencies and configurations can be packaged with the application.

3. Isolation

Different components such as the backend, AI service, and database can run in separate containers.

4. Easy Deployment

Docker Compose can be used to start multiple application services together.

5. Scalability

Individual services can be scaled independently when additional resources are required.

6. Maintainability

Separating the application into containers makes individual components easier to update and manage.

11.3 Git and Docker Together

Git and Docker complement each other in the software development process.

Git

↓

Source Code Management

↓

Versioned Project

↓

Docker

↓

Containerized Application

↓

Deployment

Git manages the project files and their versions, while Docker packages the application into reproducible environments.

11.4 Comparison of Git and Docker

Aspect	Git	Docker
Main Purpose	Version control	Containerization
Manages	Source code and changes	Application environment
Collaboration	Branches and repositories	Consistent environments
Main Unit	Commit	Container/Image
Major Benefit	Version tracking	Portability

11.5 Overall Benefits for the Proposed System

Using Git and Docker together provides the following advantages:

- Better collaboration among developers.
- Organized project version management.
- Consistent application environments.
- Easier deployment.
- Improved portability.
- Better maintainability.
- Easier recovery from errors.
- Support for future scalability.
- Reduced configuration problems.
- Better organization of the development workflow.

11.6 Summary

Git and Docker address different but complementary areas of software development. Git provides version control and collaboration, while Docker provides containerization and deployment consistency. Together, they provide a structured development and deployment approach for the proposed Smart Court Case Management and Judicial Analytics Platform.

12. Challenges and Improvements

The proposed AI-Powered Smart Court Case Management and Judicial Analytics Platform involves multiple components, including case management, document classification, analytics, database management, Git, and Docker. Several technical and organizational challenges may occur during the development and deployment of such a system.

12.1 Potential Challenges

1. Managing Multiple Services

The proposed system contains frontend, backend, AI service, and database components. Managing communication between these services can become complex.

Improvement:

Use Docker Compose to define and manage the services in a centralized configuration.

2. Git Merge Conflicts

When multiple developers modify the same files, merge conflicts may occur.

Improvement:

Use separate feature branches, meaningful commits, regular synchronization, and proper code review before merging.

3. Docker Configuration Issues

Incorrect ports, dependencies, environment variables, or network configurations may prevent containers from communicating correctly.

Improvement:

Maintain a clear Docker Compose configuration and test each service before integrating the complete environment.

4. Database Data Persistence

Containerized databases may lose data if persistent storage is not configured correctly.

Improvement:

Use Docker volumes to maintain database data even when containers are recreated.

5. Sensitive Data Protection

Court-related information may contain sensitive data that requires protection.

Improvement:

Use authentication, authorization, secure communication, encrypted storage, and appropriate access controls.

6. AI Prediction Accuracy

AI-based document classification and case-duration prediction may not always produce accurate results.

Improvement:

Use reliable datasets, regularly evaluate the models, and allow authorized users to review AI-generated results.

7. Increasing Data Volume

The number of court cases and documents may increase significantly over time.

Improvement:

Use scalable database architecture, efficient data management, indexing, and appropriate cloud infrastructure.

12.2 Git Workflow Improvements

The Git workflow can be improved by:

- Using clear branch naming conventions.
- Maintaining small and meaningful commits.
- Performing regular code reviews.
- Synchronizing branches frequently.
- Protecting the main branch.
- Using pull requests before merging.
- Maintaining release tags for stable versions.

12.3 Docker Deployment Improvements

Docker deployment can be improved by:

- Using lightweight base images.
- Keeping containers focused on individual services.
- Managing sensitive information through environment configuration.
- Using persistent volumes for important data.
- Monitoring container performance.
- Regularly updating container images.
- Using automated build and deployment pipelines.

12.4 Future Enhancements

The proposed platform can be enhanced in the future with:

1. Advanced AI Models

More advanced NLP models can be introduced for better document classification and legal text analysis.

2. Improved Case Prediction

Historical case data can be used to improve case-duration prediction.

3. Automated Scheduling

AI-based scheduling can be used to optimize judge availability, courtroom availability, and hearing priorities.

4. Advanced Analytics

More detailed dashboards can be introduced for analyzing case trends, backlog, workload, and processing time.

5. Notification System

The platform can provide automated notifications for upcoming hearings and important case events.

6. Cloud Deployment

The application can be deployed to cloud infrastructure for improved availability and scalability.

7. CI/CD Integration

Git-based continuous integration and continuous deployment pipelines can automatically test, build, and deploy application updates.

8. Enhanced Security

Future versions can implement stronger authentication, role-based access control, encryption, audit logging, and security monitoring.

12.5 Summary

The proposed system may face challenges related to service management, Git collaboration, Docker configuration, database persistence, security, AI accuracy, and scalability. These challenges can be addressed through proper architecture, version control practices, containerization, security mechanisms, and continuous improvement.

Future enhancements such as advanced AI models, automated scheduling, cloud deployment, advanced analytics, notifications, and CI/CD can further improve the functionality and reliability of the platform.